

Stack decisions awaiting sign-off

Eight architecture decision records are recorded in `technology-stack-design.md`. All are provisional: none takes effect until approved and recorded in `DECISIONS.md`, and five design documents stay blocked until that happens.

BEFORE APPROVING

The specification library has not been read in full yet, and most of these decisions are justified by what it already commits the platform to. Part two lists those commitments in plain language so they can be contested directly, rather than taken on trust. If any of them is wrong, the decision resting on it changes — and that is the more useful thing to find out.

6	3	2	5
Records needing approval	Questions only the lead can settle	Risks left open, not papered over	Design documents gated

PART ONE

The decision register

Records with an amber edge need a decision. Three diverge from the decision brief and one reverses our own earlier recommendation — those are flagged, so nothing reads as quiet agreement.

RECORD	DECISION	STATE	WHAT TO KNOW
ADR-001	Server, web and mobile stack — Node.js/TypeScript, Next.js, React Native	NEEDS APPROVAL	Server and web reaffirmed by ADR-008. Its mobile component is replaced by ADR-007 on approval, so approve 001, 007 and 008 together.
ADR-002	Review of the two stacks raised in review — SvelteKit + Capacitor, and Elixir LiveView / Rails Hotwire	CLOSED	Both evaluated and ruled out with stated tradeoffs; neither changed the recommendation. No action needed.
ADR-003	Adds four evaluation criteria — verification strength, off-the-shelf enterprise capability, dependency count, maintenance tax	ALREADY APPROVED	The criteria came from the lead — one from the standup, three from the brief — so no separate sign-off was needed. This is the change that reopened everything below.
ADR-004	Database — PostgreSQL, a typed query builder, schema-per-tenant, UUIDv7 keys	NEEDS APPROVAL	Diverges from the brief, twice Schema-per-tenant rather than shared schema with row-level security — row-level security is absent in MySQL, so it conflicts with the engine-portability directive. A query builder rather than an ORM, because mapping presupposes entities known before builders define them.
ADR-005	One deployable per product, modular monolith inside, module boundaries enforced automatically	NEEDS APPROVAL	Lands where the brief lands, but argued from the specification's own fixed dependency rules rather than adopted from it. Not continuity with the old platform.

RECORD	DECISION	STATE	WHAT TO KNOW
ADR-006	API contracts — OpenAPI at both tiers; GraphQL rejected; gRPC deferred	NEEDS APPROVAL	gRPC is deferred rather than adopted: with one deployable there is no internal service boundary for it to serve yet. It becomes the recommendation the moment a component is extracted.
ADR-007	Both a web form and a real mobile app, the builder's choice per application — mobile built in Flutter	NEEDS APPROVAL	Reverses our own earlier call Agrees with the brief on Flutter. Web-only delivery cannot satisfy the mobile capability the specification already commits to.
ADR-008	Keeps Node.js/TypeScript and Next.js for server and web, on rewritten reasoning	NEEDS APPROVAL	Diverges from the brief The brief ranks .NET first, and it does score better on four criteria — we say so plainly. We still recommend against switching because Blazor cannot serve the arbitrary published web output builders will create, so .NET's advantage only exists as a unified .NET-and-Blazor stack that is unavailable here; adopting it would take us from two languages to three, and its verification advantage is weakest on our central data path, where builders define schemas at runtime.

PART TWO

What these decisions rest on

Each line is something the specification already commits the platform to, stated plainly. These are the premises the decisions were argued from — not conclusions of their own. Any of them can be challenged, and if one is wrong the decision beside it changes.

WHAT THE SPECIFICATION ALREADY COMMITS US TO	WHAT IT DRIVES
Tenant separation must be verified at the boundary, not assumed from the absence of a reported breach — and a breach halts work rather than being logged and tolerated.	Schema-per-tenant, because a filter the code is trusted to add on every query is assumed, not verified. ADR-004
Adding one tenant must cause no measurable slowdown for any existing tenant ; the platform sustains 50,000 signed-in sessions per region and 10 million records for a single tenant.	The partitioning choice, and Risk 01 — the one number most likely to send that choice back. ADR-004, ADR-005
A change to one tenant's stored data must finish, or safely undo itself, within four hours .	Per-tenant separation, so a migration is a per-tenant operation that can be undone for one tenant alone. ADR-004
Builders define their own data structures while the platform is running — they are not known when we build it.	A query builder rather than an ORM, and API contracts the platform generates per application rather than writes in advance. ADR-004, ADR-006
Seven components and the directions they may depend on each other are fixed and cannot be changed by us unilaterally .	One deployable with those seven as internal modules, and automated checks that prove the directions hold on every change. ADR-005
A release cannot ship if we cannot prove it breaks nothing existing consumers depend on — failing to check counts as failing.	OpenAPI, because a contract a tool can compare automatically is the only kind that can satisfy this. ADR-006

A built application's **mobile form must carry every guarantee its web form carries**, with only divergence we have expressly allowed.

Flutter, whose single rendering engine makes that sameness structural rather than something each release must re-prove. ADR-007

The platform must **never specialise toward one kind of business** — every capability stays generic, and domain content belongs to builders.

Q1 in part three, and the reason several of the brief's defaults cannot be adopted as written. all records

PART THREE

Questions only the lead can settle

Each blocks work that cannot start until it is answered. Two would require reopening the frozen specification, which no design document can do.

Q1 · BLOCKS THE MOST

Was the decision brief written for the platform, or for the applications it builds?

The brief opens with "enterprise web software (HRIS, finance, inventory) plus mobile" — the exact framing AI ahaMatic is defined against, since it is a generic platform that builds software rather than being one of those things.

ASK — Several of the brief's defaults, such as append-only ledgers, as-of dating, outbox-queue sync and web-only self-service, read as choices for one business application. For the platform, each is either the builder's own content or a generic capability we would have to add to the specification. Which target was intended?

Blocks: the temporal-data question, the sync question, and part of the mobile surface decision.

Q2 · WOULD REOPEN THE SPECIFICATION

Must the platform support effective dating and append-only records for every builder?

The brief treats these as day-one requirements, on the reasoning that finance and HR are as-of domains. For a generic platform the equivalent question is whether the data-modelling capability must offer them to any builder, in any domain.

ASK — If yes, this is a specification change, not a design decision; the specification is frozen, so it goes through the change process. If it is left to builders, it stays their content and the platform core stays domain-neutral.

Why it matters: building these into the core without that step would breach the generality guarantee the platform is held to.

Q3 · WOULD REOPEN THE SPECIFICATION

Is offline sync something the platform provides, or something each application solves?

The brief makes sync posture the second most expensive decision, because it constrains the database schema. Our specification mentions offline behaviour for mobile artifacts but defines no sync model at all.

ASK — If the platform must offer a generic sync capability, that is a specification change like Q2. If it is per-application, it belongs to what builders build and not to the platform core.

Blocks: finalising the database schema — a bidirectional model would require version columns, soft deletes and a written conflict rule on everything syncable.

Known risks we did not paper over

Both follow from the decisions above and are recorded as open rather than assumed away. Either could send a decision back.

- RISK 01** **Database connections may not scale the way we need.** Schema-per-tenant combined with many running copies of the application multiplies open database connections. This is the most likely way ADR-004 and ADR-005 together could break the requirement that adding a tenant never slows down existing tenants — and if it does, the schema-per-tenant decision changes.
- RISK 02** **Flutter's device plugins are unverified.** The brief rightly asks us to check the two or three plugins we would actually depend on — who maintains them, how quickly issues are answered, whether they are first-party. That check needs the device-capability surface defined first, so ADR-007 is recommended but not yet validated.

PART FIVE

Also worth deciding while we are together

- ORDER** **Should the stack still gate everything?** The brief argues stack is the cheapest of five decisions to reverse and roughly a fifth of the outcome. The design phase is paused on exactly that decision while the expensive ones — data model, sync, architecture — sat open. We have since worked ahead on them, but the ordering question stands, and it raises whether the design library's own document ordering needs reworking.
- WORKFLOW** **Does the workflow capability adopt BPMN-style modelling?** Still an unconfirmed assumption. The only BPMN mentions in our library describe competitors, not a choice we have made — and it needs answering before that capability is designed.
- LIMITS** **Three numbers are still blank** — the retry ceiling per task, the self-correction limit before a human is called, and the token, compute and cost ceiling per session. All three were deferred to one document and never set. The middle one bears on Risk 01.
- REVIEW** **When will the specification library be read in full?** A full read might change fundamental principles and force these decisions to be revisited. Everything here is provisional until then, and Q1 suggests there may already be a difference in how the platform is being pictured.

Five of the eight records either diverge from the decision brief or reverse a recommendation we made ourselves. Each argues from the frozen specification rather than from either document, so any of them can be contested on that ground — and if the specification is wrong, that is the more useful thing to find out.